



SCHOOL OF COMPUTING SCIENCE ENGINEERING AND ARTIFICIAL INTELLIGENCE

Academic Year: Fall 2026-2027

CSE1021 Introduction to Problem Solving and Programming

PROJECT ACTIVITY - TEAM MEMBERS

Roll No	Name
26BCE10269	Vyom Sanodiya
26BCE10847	Aditya Kumar
26BCE10984	Kushagra Patel

Project Title: Mini TextDB Software

Abstract:

This project is a lightweight, file-based database system built in Python. It features a custom command parser for running queries and a clean desktop GUI that functions like a simple spreadsheet. Users can easily create tables, add typed columns, insert records, and search data either by clicking through the app or by typing out commands into a built-in terminal. It bridges structured text storage with an accessible, user-friendly workspace.

Code

engine.py:

```
import os

from enum import Enum

from src.exceptions.exceptions import (AlreadyExistException, DoesNotExistException,
EmptyException, Success)

from src.helper.helper import (get_table_path, load_table, save_table, get_storage_path)

class ColumnType(Enum):

    Integer = 1

    Decimal = 2

    String = 3

column_template = {}

def create_table(table_name: str) -> Success:

    path = get_table_path(table_name)

    if os.path.exists(path):

        raise AlreadyExistException(f"Table '{table_name}' already exists")

    os.makedirs(os.path.dirname(path), exist_ok=True)

    with open(path, "w") as file:

        import json

        json.dump({}, file, indent=4)

    return Success(f"Table {table_name} created successfully.")

def list_table():

    storage_path = get_storage_path()

    os.makedirs(storage_path, exist_ok=True)

    tables = []

    for file in sorted(os.listdir(storage_path)):
```

```

    if file.endswith(".json"):
        tables.append(file[:-5])
if len(tables) == 0:
    raise EmptyException("No tables found.")
for index, table in enumerate(tables, start=1):
    print(f"{index}. {table}")
return tables

def delete_table(table_name: str) -> Success:
    file_path = get_table_path(table_name)
    if not os.path.exists(file_path):
        raise DoesNotExistException(f"Table {table_name} could not be found.")
    os.remove(file_path)
    return Success(f"Table {table_name} deleted successfully")

def insertColumnIntoTable(table_name: str, column_name: str, column_type: ColumnType):
    checkIfTableExists(table_name)
    data = load_table(table_name)
    # Check if column already exists
    for row in data.values():
        if column_name in row:
            raise AlreadyExistException(f"Column '{column_name}' already exists.")
    # Determine default value
    if column_type == ColumnType.String:
        default_value = ""
    elif column_type == ColumnType.Integer:
        default_value = 0
    elif column_type == ColumnType.Decimal:
        default_value = 0.0

```

```

else:
    raise ValueError(f"Invalid column type: {column_type}")

# Add column to every existing row
for row in data.values():
    row[column_name] = default_value

save_table(table_name, data)

return Success(f"Column {column_name} added successfully.")

def deleteColumnFromTable(table_name: str, column_name: str):
    checkIfTableExists(table_name)
    data = load_table(table_name)
    column_exists = False
    for row in data.values():
        if column_name in row:
            del row[column_name]
            column_exists = True
    if not column_exists:
        raise DoesNotExistException(f"Column '{column_name}' does not exist.")
    save_table(table_name, data)
    return Success(f"Column {column_name} deleted successfully.")

def insertDataIntoTable(table_name: str, row_data: dict):
    checkIfTableExists(table_name)
    data = load_table(table_name)
    # Generate next numeric row ID
    if len(data) == 0:
        row_id = 1
    else:
        row_id = max(int(key) for key in data.keys()) + 1

```

```

# JSON object keys are strings
data[str(row_id)] = row_data
save_table(table_name, data)
return Success(f"Row {row_id} added successfully.")

def select_where(table_name: str, field: str, operator: str, value):
    checkIfTableExists(table_name)
    data = load_table(table_name)
    results = {}
    valid_operators = {"=", ">", ">=", "<", "<="}
    if operator not in valid_operators:
        raise ValueError(f"Invalid operator '{operator}'. "
                        f"Valid operators are: {valid_operators}")
    for row_id, details in data.items():
        if field not in details:
            continue
        field_value = details[field]
        if operator == "=":
            if field_value == value:
                results[row_id] = details
        elif operator == ">":
            if field_value > value:
                results[row_id] = details
        elif operator == ">=":
            if field_value >= value:
                results[row_id] = details
        elif operator == "<":
            if field_value < value:

```

```

        results[row_id] = details

    elif operator == "<=":

        if field_value <= value:

            results[row_id] = details

    if len(results) == 0:

        raise EmptyException("No results were found")

    return results

def select_all(table_name: str):

    checkIfTableExists(table_name)

    data = load_table(table_name)

    results = {}

    results += data.items()

    if len(results) == 0:

        raise EmptyException("No results were found")

    return results

def filtering(table_name: str, field: str):

    checkIfTableExists(table_name)

    data = load_table(table_name)

    filtered_data = []

    for row in data.values():

        if field in row:

            filtered_data.append(row[field])

    if len(filtered_data) == 0:

        raise EmptyException("No results were found")

    return filtered_data

def updateDataInTable(table_name: str, row_id: int, column_name: str, new_value):

    checkIfTableExists(table_name)

```

```

data = load_table(table_name)

row_id = str(row_id)

if row_id not in data:

    raise DoesNotExistException(f"Row '{row_id}' does not exist.")

if column_name not in data[row_id]:

    raise DoesNotExistException(f"Column '{column_name}' does not exist "

                                f"for row '{row_id}'.")

data[row_id][column_name] = new_value

save_table(table_name, data)

return Success("Data updated successfully.")

def deleteDataFromTable(table_name: str, row_id: int, column_name: str):

    checkIfTableExists(table_name)

    data = load_table(table_name)

    row_id = str(row_id)

    if row_id not in data:

        raise DoesNotExistException(f"Row '{row_id}' does not exist.")

    if column_name not in data[row_id]:

        raise DoesNotExistException(f"Column '{column_name}' does not exist "

                                    f"for row '{row_id}'.")

    del data[row_id][column_name]

    save_table(table_name, data)

    return Success("Data deleted successfully.")

def checkIfTableExists(table_name: str):

    if not os.path.exists(get_table_path(table_name)):

        raise DoesNotExistException(f"Table '{table_name}' does not exist.")

    return True

def execute(operation):

```

```

try:
    result = operation()
    if result is not None:
        print(result)
    return result
except (AlreadyExistsException, DoesNotExistException, EmptyException, ValueError) as e:
    print(e)
    return None

if __name__ == '__main__':
    # Create
    execute(lambda: create_table("students"))

    # Insert first row
    execute(lambda: insertDataIntoTable("students", {"name": "Aditya"}))

    # Add columns
    execute(lambda: insertColumnIntoTable("students", "age", ColumnType.Integer))
    execute(lambda: insertColumnIntoTable("students", "percentage", ColumnType.Decimal))

    # Insert more rows
    execute(lambda: insertDataIntoTable("students", {"name": "Samit", "age": 19, "percentage": 87.5}))
    execute(lambda: insertDataIntoTable("students", {"name": "Dhruv", "age": 17, "percentage": 92.3}))

    # Update using row ID
    execute(lambda: updateDataInTable("students", 1, "age", 18))
    execute(lambda: updateDataInTable("students", 1, "percentage", 91.2))

    # Select column
    execute(lambda: filtering("students", "age"))

    # WHERE
    execute(lambda: select_where("students", "age", ">", 18))

```

```

# Delete value from row

execute(lambda: deleteDataFromTable("students", 3, "percentage"))

# Delete column

execute(lambda: deleteColumnFromTable("students", "percentage"))

# List tables

execute(lambda: list_table())

# Delete table # execute(lambda: delete_table("students"))

```

parser.py:

```

import ast
import json
import re

from engine import (
    ColumnType, create_table, list_table, delete_table, insertColumnIntoTable,
    deleteColumnFromTable, insertDataIntoTable, select_where, select_all, filtering,
    updateDataInTable, deleteDataFromTable, checkIfTableExists
)

from src.helper.helper import load_table, save_table
from src.exceptions.exceptions import EmptyException

class DatabaseCommandParser:

    def parse_val(self, val_str):
        val_str = val_str.strip()

        try:
            return ast.literal_eval(val_str)

```

except Exception:

lv = val_str.lower()

if lv == 'true': return True

if lv == 'false': return False

if lv == 'null': return None

return val_str

def execute_command(self, cmd):

try:

res = self._parse_and_run(cmd)

if res is None: return "Command executed successfully."

if hasattr(res, "message"): return res.message

if isinstance(res, dict):

if not res: return "No results."

return "\n".join([f"ID: {k} | {json.dumps(v)}" for k, v in res.items()])

if isinstance(res, list):

if not res: return "No results."

return "\n".join([f"{i+1}. {v}" for i, v in enumerate(res)])

return str(res)

except Exception as e:

return f"Error: {e}"

def _parse_and_run(self, cmd):

cmd = cmd.strip()

if not cmd: return "Command empty."

```
m = re.match(r"^CREATE\s+TABLE\s+([\w\.-]+)$", cmd, re.I)
```

```
if m: return create_table(m.group(1))
```

```
if re.match(r"^SHOW\s+TABLES$", cmd, re.I):
```

```
    return list_table()
```

```
m = re.match(r"^DROP\s+TABLE\s+([\w\.-]+)$", cmd, re.I)
```

```
if m: return delete_table(m.group(1))
```

```
m = re.match(r"^DROP\s+COLUMN\s+([\w\.-]+)\s+([\w\.-]+)$", cmd, re.I)
```

```
if m: return deleteColumnFromTable(m.group(1), m.group(2))
```

```
m = re.match(r"^ADD\s+COLUMN\s+([\w\.-]+)\s+([\w\.-]+)\s+([\w\.-]+)$", cmd, re.I)
```

```
if m:
```

```
    table, col, ctype = m.groups()
```

```
    checkIfTableExists(table)
```

```
    data = load_table(table)
```

```
    if len(data) == 0:
```

```
        data["0"] = {}
```

```
        save_table(table, data)
```

```
enum_type = ColumnType.String
```

```
ctype_u = ctype.upper()
```

```
if ctype_u in ["INTEGER", "INT"]: enum_type = ColumnType.Integer
```

```
elif ctype_u in ["DECIMAL", "FLOAT"]: enum_type = ColumnType.Decimal
```

```
return insertColumnIntoTable(table, col, enum_type)
```

```
m = re.match(r"^INSERT\s+INTO\s+([\w\~]+)\s+VALUES\s+(.+)$", cmd, re.I)
```

```
if m:
```

```
    table, val_str = m.groups()
```

```
    row_data = ast.literal_eval(val_str.strip())
```

```
    return insertDataIntoTable(table, row_data)
```

```
m = re.match(r"^SELECT\s+\*\s+FROM\s+([\w\~]+)\s+WHERE\s+([\w\~]+)\s*(>|=|<|=|>|<)\s*(.+)$", cmd, re.I)
```

```
if m:
```

```
    table, field, op, val = m.groups()
```

```
    res = select_where(table, field, op, self.parse_val(val))
```

```
    if "0" in res: del res["0"]
```

```
    if not res: raise EmptyException("No results were found")
```

```
    return res
```

```
m = re.match(r"^SELECT\s+([\w\~]+)\s+FROM\s+([\w\~]+)$", cmd, re.I)
```

```
if m:
```

```
    col, table = m.groups()
```

```
    if col == "*":
```

```
        res = select_all(table)
```

```
        if "0" in res: del res["0"]
```

```
        if not res: raise EmptyException("No results were found")
```

```
        return res
```

```
else:
```

```
    checkIfTableExists(table)
```

```
data = load_table(table)
```

```
filtered = [row[col] for rid, row in data.items() if rid != "0" and col in row]
```

```
if not filtered: raise EmptyException("No results were found")
```

```
return filtered
```

```
m = re.match(r"^UPDATE\s+([\w\.-]+)\s+SET\s+([\w\.-]+\s*=\s*(.+?)\s+WHERE\s+id\s*=\s*(\d+)\s*$", cmd, re.I)
```

```
if m:
```

```
    table, col, val_str, rid = m.groups()
```

```
    return updateDataInTable(table, int(rid), col, self.parse_val(val_str))
```

```
m = re.match(r"^DELETE\s+FROM\s+([\w\.-]+)\s+COLUMN\s+([\w\.-]+\s+WHERE\s+id\s*=\s*(\d+)\s*$", cmd, re.I)
```

```
if m:
```

```
    table, col, rid = m.groups()
```

```
    return deleteDataFromTable(table, int(rid), col)
```

```
raise ValueError("Syntax Error: Unknown or malformed command.")
```

gui.py:

```
import tkinter as tk
```

```
from tkinter import ttk, messagebox
```

```
import json
```

```
from db_parser import DatabaseCommandParser
```

```
from src.helper.helper import load_table, save_table
```

```
class TextDBApp:
```

```
    def __init__(self, root):
```

```
        self.root = root
```

```
        self.root.title("Mini Text Database - Desktop App")
```

```
        self.root.geometry("950x700")
```

```
        self.current_table = tk.StringVar()
```

```
        self.parser = DatabaseCommandParser()
```

```
        self.setup_ui()
```

```
    def setup_ui(self):
```

```
        self.sidebar = tk.Frame(self.root, width=200, bg="#e0e0e0", relief="sunken", bd=2)
```

```
        self.sidebar.pack(side="left", fill="y")
```

```
        tk.Label(self.sidebar, text="Active Table:", bg="#e0e0e0", font=("Arial", 10, "bold")).pack(pady=(20, 5))
```

```
        self.table_selector = ttk.Combobox(self.sidebar, textvariable=self.current_table, state="readonly")
```

```
        self.table_selector.pack(padx=10, fill="x")
```

```
        self.table_selector.bind("<<ComboboxSelected>>", lambda e: self.render_list_view())
```

```
tk.Button(self.sidebar, text="Create Table",  
command=self.open_create_table_modal).pack(pady=(20, 5), padx=10, fill="x")
```

```
tk.Button(self.sidebar, text="Add Column",  
command=self.open_add_column_modal).pack(pady=5, padx=10, fill="x")
```

```
tk.Button(self.sidebar, text="Create Record",  
command=self.open_create_modal).pack(pady=5, padx=10, fill="x")
```

```
tk.Button(self.sidebar, text="List Records", command=self.render_list_view).pack(pady=5,  
padx=10, fill="x")
```

```
tk.Button(self.sidebar, text="Delete Table", command=self.delete_current_table,  
fg="red").pack(pady=(25, 5), padx=10, fill="x")
```

```
self.main_area = tk.Frame(self.root, bg="white")
```

```
self.main_area.pack(side="right", fill="both", expand=True)
```

```
self.query_frame = tk.Frame(self.main_area, bg="#f8f8f8", bd=1, relief="solid")
```

```
self.query_frame.pack(fill="x", padx=10, pady=10)
```

```
tk.Label(self.query_frame, text="Search:", bg="#f8f8f8").pack(side="left", padx=5, pady=5)
```

```
self.search_field = ttk.Combobox(self.query_frame, width=15, state="readonly")
```

```
self.search_field.pack(side="left", padx=5, pady=5)
```

```
self.search_op = ttk.Combobox(self.query_frame, width=5, values=["=", ">", "<", ">=", "<="],  
state="readonly")
```

```
self.search_op.pack(side="left", padx=5, pady=5)
```

```
self.search_op.set("=")
```

```
self.search_val = tk.Entry(self.query_frame, width=15)
```

```
self.search_val.pack(side="left", padx=5, pady=5)
```

```
tk.Button(self.query_frame, text="Search", command=self.perform_search).pack(side="left",  
padx=5, pady=5)
```

```
tk.Button(self.query_frame, text="Reset", command=self.render_list_view).pack(side="left",  
padx=5, pady=5)
```

```

self.cli_frame = tk.Frame(self.main_area, bg="#1e1e1e", bd=2, relief="sunken")

self.cli_frame.pack(side="bottom", fill="x", padx=10, pady=10)

tk.Label(self.cli_frame, text="Terminal Output:", bg="#1e1e1e", fg="#a0a0a0",
font=("Consolas", 8)).pack(anchor="w", padx=5, pady=(2, 0))

self.cli_output = tk.Text(self.cli_frame, height=5, bg="#1e1e1e", fg="#4af626",
font=("Consolas", 10), state="disabled", bd=0)

self.cli_output.pack(fill="x", padx=5, pady=2)


input_frame = tk.Frame(self.cli_frame, bg="#1e1e1e")

input_frame.pack(fill="x", padx=5, pady=(0, 5))

tk.Label(input_frame, text="TextDB> ", bg="#1e1e1e", fg="ffffff", font=("Consolas", 10,
"bold")).pack(side="left")

self.cli_input = tk.Entry(input_frame, bg="#1e1e1e", fg="ffffff", font=("Consolas", 10),
insertbackground="white", bd=0)

self.cli_input.pack(side="left", fill="x", expand=True)

self.cli_input.bind("<Return>", self.handle_cli_command)


btn_frame = tk.Frame(self.main_area, bg="white")

btn_frame.pack(side="bottom", fill="x", padx=10, pady=(0, 5))

tk.Button(btn_frame, text="✎ Edit Selected Record",
command=self.open_edit_modal).pack(side="left", padx=5)

tk.Button(btn_frame, text="🗑 Delete Selected Record",
command=self.delete_selected_record, fg="red").pack(side="right", padx=5)


self.tree_frame = tk.Frame(self.main_area)

self.tree_frame.pack(fill="both", expand=True, padx=10, pady=5)

self.tree = ttk.Treeview(self.tree_frame, show="headings")

self.tree.pack(side="left", fill="both", expand=True)

scrollbar = ttk.Scrollbar(self.tree_frame, orient="vertical", command=self.tree.yview)

scrollbar.pack(side="right", fill="y")

```

```

self.tree.configure(yscrollcommand=scrollbar.set)

self.refresh_table_list()

def handle_cli_command(self, event):

    cmd_str = self.cli_input.get().strip()

    if not cmd_str: return

    self.cli_input.delete(0, tk.END)

    self.log_to_cli(f"TextDB> {cmd_str}")

    output = self.parser.execute_command(cmd_str)

    self.log_to_cli(str(output))

    self.refresh_table_list()

    self.render_list_view()

def log_to_cli(self, message):

    self.cli_output.config(state="normal")

    self.cli_output.insert(tk.END, message + "\n")

    self.cli_output.see(tk.END)

    self.cli_output.config(state="disabled")

def refresh_table_list(self):

    try:

        tables_output = self.parser.execute_command("SHOW TABLES")

        if "Error" not in tables_output and "No results" not in tables_output:

            tables = [t.split(" ")[1] for t in tables_output.split("\n") if ". " in t]

            self.table_selector["values"] = tables

            if tables and not self.current_table.get():

```

```
        self.table_selector.current(0)

        self.render_list_view()
    else:
        self.table_selector["values"] = []

        self.current_table.set("")

        self.render_list_view()
except Exception:
    pass
```

```
def render_list_view(self, event=None, search_results=None):
```

```
    for item in self.tree.get_children():
        self.tree.delete(item)
```

```
    table = self.current_table.get()
```

```
    if not table:
```

```
        self.tree["columns"] = ()
```

```
        return
```

```
    try:
```

```
        data = search_results if search_results is not None else load_table(table)
```

```
        if "0" in data:
```

```
            columns = list(data["0"].keys())
```

```
            del data["0"]
```

```
        else:
```

```
            columns = []
```

```
            for details in data.values():
```

```
                for col in details.keys():
```

```
                    if col not in columns: columns.append(col)
```

```
all_cols = ["ID"] + columns
self.tree["columns"] = all_cols
self.search_field["values"] = columns
```

```
for col in all_cols:
    self.tree.heading(col, text=col)
    self.tree.column(col, width=100, anchor="w")
```

```
for row_id, details in data.items():
    row_values = [row_id] + [details.get(col, "") for col in columns]
    self.tree.insert("", "end", iid=row_id, values=row_values)
```

```
except Exception:
```

```
    self.tree["columns"] = ()
```

```
def perform_search(self):
```

```
    table = self.current_table.get()
```

```
    field = self.search_field.get()
```

```
    op = self.search_op.get()
```

```
    val_str = self.search_val.get()
```

```
    if not table or not field or not val_str: return
```

```
    cmd = f"SELECT * FROM {table} WHERE {field} {op} {val_str}"
```

```
    self.log_to_cli(f"TextDB> {cmd}")
```

```
    try:
```

```
        res = self.parser._parse_and_run(cmd)
```

```
        self.render_list_view(search_results=res)
```

```

        self.log_to_cli("Query executed successfully.")

    except Exception as e:

        self.render_list_view(search_results={})

        self.log_to_cli(f"Error: {e}")


def open_add_column_modal(self):

    table = self.current_table.get()

    if not table: return


    modal = tk.Toplevel(self.root)
    modal.title("Add Column")
    modal.geometry("250x200")
    modal.grab_set()


    tk.Label(modal, text="Column Name:").pack(pady=(10, 0))

    entry_name = tk.Entry(modal)
    entry_name.pack(pady=5)


    tk.Label(modal, text="Data Type:").pack(pady=(10, 0))

    type_combo = ttk.Combobox(modal, values=["STRING", "INTEGER", "DECIMAL"],
state="readonly")

    type_combo.set("STRING")
    type_combo.pack(pady=5)


def save_column():

    col_name = entry_name.get().strip().replace(" ", "_")

    if not col_name: return

    cmd = f"ADD COLUMN {table} {col_name} {type_combo.get().strip()}"

```

```
output = self.parser.execute_command(cmd)
```

```
self.log_to_cli(f"TextDB> {cmd}")
```

```
self.log_to_cli(str(output))
```

```
self.render_list_view()
```

```
modal.destroy()
```

```
tk.Button(modal, text="Add", command=save_column).pack(pady=10)
```

```
def open_create_modal(self):
```

```
    table = self.current_table.get()
```

```
    if not table: return
```

```
    try:
```

```
        data = load_table(table)
```

```
    except Exception:
```

```
        data = {}
```

```
    columns = []
```

```
    if "0" in data:
```

```
        columns = list(data["0"].keys())
```

```
    else:
```

```
        for details in data.values():
```

```
            for col in details.keys():
```

```
                if col not in columns: columns.append(col)
```

```
    if not columns:
```

```
        messagebox.showwarning("No Columns", "This table has no columns yet.\nPlease click  
'Add Column' to define the format first!")
```

```
return
```

```
modal = tk.Toplevel(self.root)
```

```
modal.title("Create Record")
```

```
modal.geometry("350x400")
```

```
modal.grab_set()
```

```
entries = {}
```

```
tk.Label(modal, text="Enter Record Details:").pack(pady=(10, 0))
```

```
for col in columns:
```

```
    tk.Label(modal, text=f"{col}:").pack(pady=(5, 0))
```

```
    ent = tk.Entry(modal, width=30)
```

```
    ent.pack()
```

```
    entries[col] = ent
```

```
def save():
```

```
    row_dict = {col: self.parser.parse_val(ent.get().strip()) for col, ent in entries.items()}
```

```
    cmd = f"INSERT INTO {table} VALUES {json.dumps(row_dict)}"
```

```
    output = self.parser.execute_command(cmd)
```

```
    self.log_to_cli(f"TextDB> {cmd}")
```

```
    self.log_to_cli(str(output))
```

```
    self.render_list_view()
```

```
    modal.destroy()
```

```
tk.Button(modal, text="Save Record", command=save).pack(pady=15)
```

```
def open_edit_modal(self):
```

```
selected_item = self.tree.selection()

if not selected_item: return

table = self.current_table.get()

record_id = selected_item[0]

row_values = self.tree.item(record_id)['values']
```

```
try:

    data = load_table(table)

    columns = list(data[record_id].keys())

except Exception: return
```

```
modal = tk.Toplevel(self.root)

modal.title(f"Edit Record ID: {record_id}")

modal.geometry("350x400")

modal.grab_set()
```

```
entries = {}

for i, col in enumerate(columns):

    tk.Label(modal, text=f"{col}:").pack(pady=(5, 0))

    ent = tk.Entry(modal, width=30)

    ent.insert(0, str(row_values[i+1]))

    ent.pack()

    entries[col] = ent
```

```
def save_edits():

    try:

        for col, ent in entries.items():
```

```

        val = self.parser.parse_val(ent.get().strip())

        self.parser.execute_command(f"UPDATE {table} SET {col} = {repr(val)} WHERE id = {record_id}")

        self.render_list_view()

        modal.destroy()

    except Exception as e: messagebox.showerror("Error", str(e))

```

```

tk.Button(modal, text="Save Edits", command=save_edits).pack(pady=15)

```

```

def delete_selected_record(self):
    selected_item = self.tree.selection()

    if not selected_item: return

    record_id = selected_item[0]

    if messagebox.askyesno("Confirm", f"Delete row ID {record_id}?"):

        try:
            data = load_table(self.current_table.get())

            if record_id in data:
                del data[record_id]

                save_table(self.current_table.get(), data)

            self.render_list_view()

        except Exception as e: messagebox.showerror("Error", str(e))

```

```

def open_create_table_modal(self):
    modal = tk.Toplevel(self.root)

    modal.title("Create Table")

    modal.geometry("250x150")

    modal.grab_set()

```

```
tk.Label(modal, text="Table Name:").pack(pady=(20, 5))
```

```
entry_table = tk.Entry(modal)
```

```
entry_table.pack(pady=5)
```

```
def save_table_action():
```

```
    t_name = entry_table.get().strip().replace(" ", "_")
```

```
    if not t_name: return
```

```
    cmd = f"CREATE TABLE {t_name}"
```

```
    output = self.parser.execute_command(cmd)
```

```
    self.log_to_cli(f"TextDB> {cmd}")
```

```
    self.log_to_cli(str(output))
```

```
    self.refresh_table_list()
```

```
    modal.destroy()
```

```
tk.Button(modal, text="Create", command=save_table_action).pack(pady=10)
```

```
def delete_current_table(self):
```

```
    table_name = self.current_table.get()
```

```
    if not table_name: return
```

```
    if messagebox.askyesno("Delete Table", f"Delete '{table_name}' entirely?"):
```

```
        cmd = f"DROP TABLE {table_name}"
```

```
        output = self.parser.execute_command(cmd)
```

```
        self.log_to_cli(f"TextDB> {cmd}")
```

```
        self.log_to_cli(str(output))
```

```
        self.current_table.set("")
```

```
        self.refresh_table_list()
```

```
        self.render_list_view()
```

```
if __name__ == "__main__":  
    root = tk.Tk()  
    app = TextDBApp(root)  
    root.mainloop()
```

Output

Mini Text Database - Desktop App

Active Table: Table_1

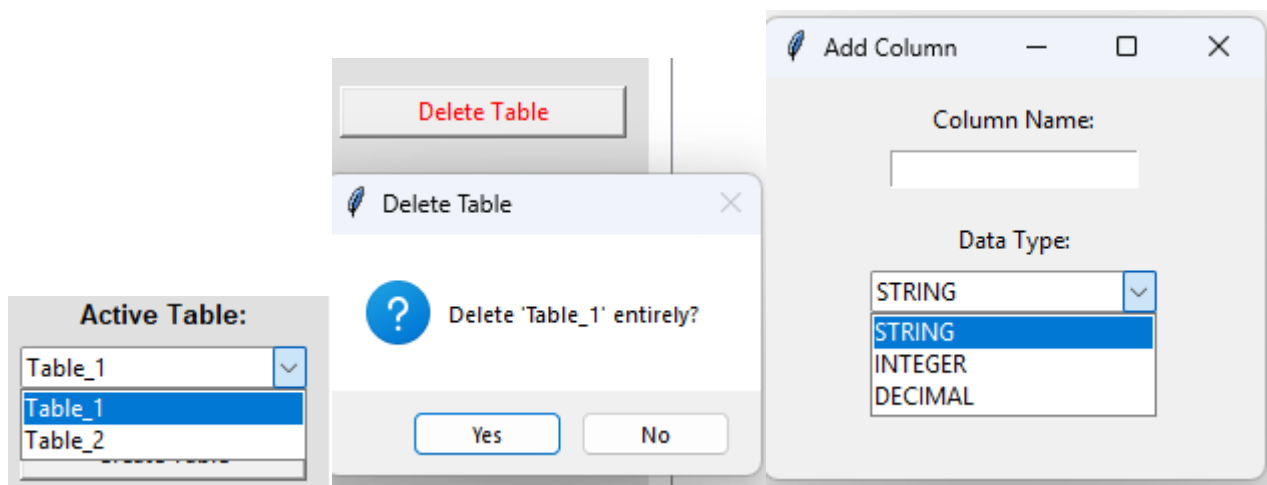
Create Table
Add Column
Create Record
List Records
Delete Table

Search: [] [] Search Reset

ID	Name	Age	Branch
1	Vyom	19	CSE
2	Aditya	18	CSE
3	Kushagra	18	CSE

Edit Selected Record Delete Selected Record

Terminal Output:
TextDB> INSERT INTO Table_1 VALUES ("Name": "Aditya", "Age": 18, "Branch": "CSE")
Row 2 added successfully.
TextDB> INSERT INTO Table_1 VALUES ("Name": "Kushagra", "Age": 18, "Branch": "CSE")
Row 3 added successfully.
TextDB>



 Create Record—□×

Enter Record Details:

Name:

Age:

Branch:

Save Record

```
=====
      TEXT DATABASE CONSOLE
=====
Type EXIT or QUIT to close the database.
Type HELP to see supported commands.

TextDB> list tables
Error: Unknown command 'LIST'.
TextDB> help

Supported commands:

CREATE TABLE students
SHOW TABLES
DROP TABLE students

ADD COLUMN students name STRING
DROP COLUMN students name

INSERT INTO students VALUES {"name": "Kushagra", "age": 18}

SELECT * FROM students
SELECT name FROM students
SELECT * FROM students WHERE age >= 18

UPDATE students SET age = 19 WHERE id = 1

DELETE FROM students COLUMN age WHERE id = 1

EXIT

TextDB> show tables
```

```
TextDB> show tables
1. students
TextDB> create table students
Error: Table 'students' already exists
TextDB> create table teachers
Table teachers created successfully.
TextDB> insert into teachers values {"name": "Kumarasamy Sir", "comment": "best teacher"}
Row 1 added successfully.
TextDB> select * from teachers
ID: 1 | {"name": "Kumarasamy Sir", "comment": "best teacher"}
TextDB>
```